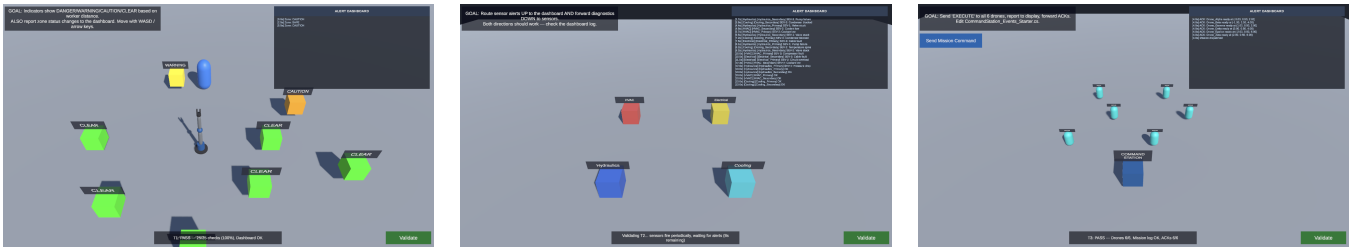
A Messaging Toolkit for Spatial Hierarchical Routing in Interactive Applications

Anonymous Author(s)



(a) Safety Zone Alerts (HRI) (b) Sensor Alert Aggregation (IoT) (c) Drone Swarm (Command & Control)

Figure 1: Cascade enables declarative spatial and hierarchical message routing for interactive applications. Study participant views during coding tasks, each with an alert dashboard (upper right), feedback (upper left), and validation UI (bottom); participant can also send additional commands to test their code. (a) Robot arm workspace where proximity alerts propagate only to indicators within specified distance thresholds (`Within()`). (b) IoT sensor nodes notify alerts upward through subsystem hierarchy (`NotifyValue`), while diagnostic commands broadcast downward (`ToDescendants`). (c) Command station broadcasts mission orders to nearby drones and collects per-drone acknowledgments back up through routing hierarchy.

Abstract

Interactive applications built on hierarchical scene graphs often rely on complex inter-component communication. Many frameworks for developing these applications support message communication up and down the parent–child hierarchy, but require custom code to define non-hierarchical communication graphs. For example, this includes sending messages between objects based on their spatial proximity, independent of the scene-graph hierarchy. Furthermore, limited prior work has empirically evaluated whether frameworks that support non-hierarchical message routing improve developer productivity over custom code. We present Cascade, a messaging toolkit for hierarchical scene graphs that integrates composable spatial filtering primitives (proximity, direction, bounding region) directly into message routing. Unlike previous work, it enables developers to create non-hierarchical communication graphs that express complex routing patterns in a single chained expression. We evaluate Cascade through a controlled within-subjects experiment comparing task completion rate, code correctness, cognitive load, and usability for Cascade and event-based messaging across tasks in robot teleoperation, Internet of Things (IoT), and command & control domains.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; *Mixed / augmented reality*; • **Software and its engineering** → **Domain specific languages**; *API languages*; • **Computing methodologies** → Robotic planning.

Submitted for review,
2026.

Keywords

domain-specific language, message routing, scene graph, spatial filtering, fluent API, developer tools, human-robot interaction, user study

ACM Reference Format:
Anonymous Author(s). 2026. A Messaging Toolkit for Spatial Hierarchical Routing in Interactive Applications. In *Proceedings of (Submitted for review).* ACM, New York, NY, USA, 17 pages.

1 Introduction

Consider a developer building an eXtended Reality (XR) user interface (UI) for human–robot interaction (HRI) (Figure 1a). When a robot's confidence drops below a threshold, nearby status panels should display warnings, spatial indicators should highlight the affected robot, and feeds from unrelated robots should remain unaffected. Implementing this with traditional event-based messaging requires manually iterating through recipients, calculating distances, and filtering by state, resulting in verbose, error-prone code duplicated for each awareness pattern.

This challenge pervades interactive applications built on hierarchical scene graphs for XR, HRI, simulations, smart environment systems, and other domains in which game engines are increasingly adopted [9]. Existing approaches, from Unity [68] Events to publish/subscribe (pub/sub) libraries, lack either hierarchy awareness, spatial filtering, or both [10, 11, 15, 67], and the R&R pattern [14, 15], which introduced hierarchy-aware routing, requires opaque metadata blocks for non-default routing, limiting discoverability.

Despite this range of approaches, two fundamental gaps persist. First, spatial predicates are well established for network-level interest management [62], Internet of Things (IoT) subscriptions [18], and collaborative awareness models [3, 37], but have not been integrated into intra-application messaging; existing workarounds all

separate spatial reasoning from message dispatch, forcing developers to write manual distance loops for every proximity-based pattern [57]. Second, no prior work has empirically evaluated whether integrating spatial predicates into a messaging Application Programming Interface (API) would actually improve developer productivity over these manual approaches [14, 15, 44].

Since hierarchical scene graphs encode both logical hierarchy and spatial relationships, we believe that a messaging API should exploit both. By providing an internal Domain-Specific Language (DSL) [20] with composable spatial filtering, we can reduce boilerplate while enabling location-aware delivery that would otherwise require significant custom code. The need for such an API is particularly acute in XR and HRI systems, in which entities are spatially distributed by design and proximity relationships directly determine interaction semantics (e.g., safety zones around robots, spatial audio attenuation, and context-dependent UI visibility).

To address this, we present Cascade, a fluent DSL [19] for hierarchical message routing in scene-graph–based applications (Figure 2). Each method in a chain represents a composable routing decision discoverable through Integrated Development Environment (IDE) autocompletion, while spatial filters route messages by proximity, direction, and bounding regions without manual distance calculations. Cascade includes static analyzers for compile-time error detection [40] and a standard library of pre-built message types for common interface and input patterns. While implemented in Unity 6.3 LTS, the design is applicable to any framework with a hierarchical scene graph. To evaluate our approach, we conducted a controlled within-subjects experiment comparing Cascade against traditional event-based messaging across tasks spanning robot teleoperation and IoT monitoring domains.

Thus, we make the following contributions:

- **C1: Composable spatial message routing.** Spatial filters (proximity, cone, direction, bounding region) that bring spatial predicates, established in network-level and awareness research [3, 37], into an intra-application message routing API as composable, typed primitives.
- **C2: Fluent internal DSL for hierarchical messaging.** A self-documenting API in which each method in a chain represents a composable routing decision (hierarchy traversal, state filtering, spatial predicates), discoverable through IDE autocompletion, complemented by static analyzers for compile-time error detection and a standard library of pre-built message types.
- **C3: First empirical evaluation of intra-application spatially-aware message routing.** A controlled within-subjects experiment comparing code correctness, cognitive load, and usability between Cascade and event-based messaging in robot teleoperation and IoT monitoring tasks.

2 Related Work

Inter-Component Communication in Game Engines. Unity provides several built-in messaging mechanisms: UnityEvents enable Inspector-configurable callbacks but require manual wiring without hierarchical propagation [67]; C# events create compile-time dependencies; and `SendMessage`/`BroadcastMessage` support hierarchy-based delivery but incur reflection overhead [15]. ECS architectures [17, 51] organize data and behavior into components

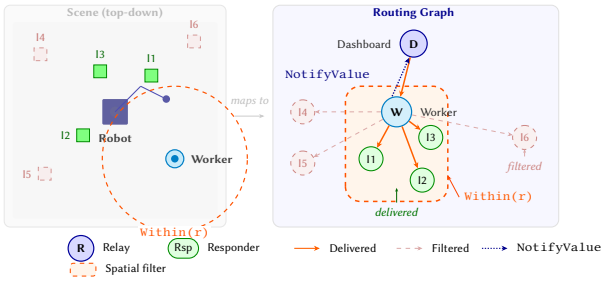


Figure 2: Cascade system architecture. Relay nodes (blue) route messages through explicit routing graph that can mirror or extend scene-graph hierarchy. Responder nodes (green) handle messages at leaf positions. Spatial filters (orange halos) restrict delivery by distance or direction. Arrows show message propagation paths for `Within(2f)` spatial query.

and systems but do not provide built-in inter-component messaging. Unity's ECS implementation (DOTS) targets high-performance data processing rather than developer-facing communication APIs.

MessagePipe [10] provides high-performance pub/sub and R3 [11] offers reactive event processing; neither supports hierarchy-aware routing or spatial filtering. Marum et al. [44] benchmarked dependency-graph reactivity against native Unity and UniRx, but evaluated performance rather than developer experience. The R&R pattern [14, 15] addresses coupling through relay nodes with hierarchy-aware routing, but its API requires explicit enum-based metadata objects for non-default routing and provides no spatial filtering.

Godot 3.x included a `ProximityGroup` node that broadcast to nearby nodes using grid-cell spatial hashing, but it supported only cuboid regions, could not compose spatial predicates with hierarchy direction or message type, and was removed in Godot 4.0 with no equivalent proximity-broadcasting node (the suggested `VisibleOnScreenNotifier3D` detects camera frustum visibility, not inter-object proximity). While spatial predicates are used in network interest management [62] and IoT middleware, none of these intra-application frameworks integrates composable spatial predicates into its message dispatch API. Cascade addresses this gap.

DSL Effectiveness and API Usability. Controlled studies have evaluated DSL effectiveness, finding that domain scientists completed tasks 31–56% faster with a DSL than with C++ [36], with the strongest gains for less experienced developers [31, 36], improved correctness with visual scripting in game engines [66], and performance gains for certain task types with a REST DSL over manual implementation [56]. Heltweg et al. [29] found that Jayvee, a DSL for data engineering, improved pipeline construction speed and quality over general-purpose Python, while Strodick and Schattkowsky [63] compared third-party visual scripting frameworks in Unity across task domains including dialog systems and NPC behavior.

Mehlhorn and Hanenberg [46] compared code understandability of imperative loops against Java's Stream API; Reichl et al. [52] found that a specialized stream debugger improved performance overall, though the benefit was moderated by task characteristics. Studies of API learnability [64, 65] show that method placement

significantly affects developer success, while Ellis et al. [13] established methodology for comparing API alternatives through timed coding tasks ($p = 0.005$ for object construction time).

We draw on several frameworks for evaluating API design. Green and Petre's Cognitive Dimensions [26] provide vocabulary for API usability; Myers and Stylos [47] synthesized practical guidelines; Piccioni et al. [50] identified common API usability obstacles; and Ko et al. [39] identified learning barriers in end-user programming. Salvaneschi et al. [53, 54] compared reactive programming against the observer pattern, finding improved correctness for comprehension tasks; their approach of comparing communication paradigms through timed coding tasks is the closest precedent to our study. Zimmerle and Gama [72] found that reactive programming APIs, which model data flow as observable streams transformed by functional operators (e.g., UniRx [11], MessagePipe [10]), impose usability burdens due to their reliance on functional programming abstractions, supporting our choice to avoid reactive-style APIs in Cascade's DSL. Following Olsen's framework [48], we evaluate Cascade's expressive leverage through task-based comparison, complemented by an unweighted NASA-TLX [28] and SUS [5].

Spatial Interaction and Proximity-Based Systems. Benford and Fahlén's spatial model [3] introduced aura, nimbus, and focus for mediating awareness in collaborative virtual environments, establishing spatial predicates as fundamental to multi-entity interaction. Julier et al. [37] applied distance, angle, and region-based filtering to manage information density in mobile XR. MacIntyre's Repo-3D [42] introduced *local variations*, allowing parts of a shared distributed scene graph to be locally added, removed, or replaced per node, a mechanism relevant to the asymmetric topologies we analyze in Section 3.8.

At the network level, spatial pub/sub systems [62] apply spatial predicates to state dissemination for multiplayer games, and FIWARE's NGSI-v2 context broker supports geographic proximity subscriptions for IoT devices [18]. Unreal Engine's Gameplay Ability System supports spatial target collection (radius, cone), but target selection and message dispatch are separate operations.

Interactive Application Frameworks and Networked XR. From early toolkits such as DART [43] to recent XR authoring systems [7, 12, 30, 33, 35, 69, 71], tools have addressed inter-device communication through visual and spatial programming but do not provide a general-purpose intra-application messaging layer.

Research on asymmetric collaboration [27, 38] shows that different users require different information modalities; systems like Ubiq [21], VirtualNexus [34], and Spacetime [70] support heterogeneous user experiences across shared environments, while Sereno et al. [59] survey design considerations for collaborative XR including role and technology symmetry. Cascade extends these concepts with unified local/network routing and analysis of message propagation under graph asymmetry (Section 3.8).

HRI and Teleoperation UIs. Unity has become a common platform for HRI systems [9], with ROS [49] handling inter-process communication [23, 61] and Unity handling the operator UI. While ROS has documented learning barriers [6], communication *within* the Unity application still relies on ad-hoc event wiring. Recent work on assembly goal specification [1] and shared autonomy [41] demonstrates multi-level, inherently asymmetric interaction patterns involving communication between goal specification, monitoring, and control layers. Cascade's directional routing and spatial filtering address these intra-application needs, complementing inter-process frameworks such as ROS.

```
// (a) Original R&R API
relay.Invoke(Method.MessageString, "Hello");
var md = new MetadataBlock(LevelFilter.Descendants,
  ActiveFilter.Active, SelectedFilter.All, NetworkFilter.Local);
relay.Invoke(Method.MessageString, "Hello", md);

// (b) Property routing (auto-executes)
relay.To.Send("Hello");
relay.To.Descendants.Active.Send("Hello");

// (c) Fluent chain (explicit Execute)
relay.Send("Hello").Execute();
relay.Send("Hello").ToDescendants().Active().Within(10f).Execute();
```

Figure 3: Three syntaxes for same task. (a) Original API requires opaque enumeration values for non-default routing. (b) Property routing auto-executes with most concise syntax. (c) Fluent chain adds composable spatial filtering; explicit `Execute()` call makes dispatch point visible in code review and prevents accidental side effects during chain construction.

3 System Design

Cascade adopts the R&R routing architecture [14, 15] (Figure 2) and layers a fluent DSL on top, adds spatial filtering primitives absent from existing intra-application messaging frameworks, and introduces static analysis tooling and a standard message library. These additions address the first gap noted in Section 1 through composable spatial routing integrated directly into message dispatch. The second gap, empirical validation, is addressed through our user study (Section 5). While our implementation targets Unity, the design is applicable to any hierarchical scene graph.

3.1 Design Goals

We designed Cascade's API around four goals derived from prior work on API usability [26, 65]:

- **Conciseness**: Replace opaque metadata construction with readable method calls.
- **Discoverability**: Each method returns a builder whose available methods represent valid next steps, enabling IDE autocompletion.
- **Composability**: Hierarchy, state, and spatial filters combine arbitrarily without restrictions.
- **Type Safety**: Invalid filter combinations or missing terminal operations produce compiler errors, not silent failures.

3.2 Fluent DSL

Because the DSL is embedded in C#, every routing expression is a valid statement that benefits from IDE autocompletion, type checking, and refactoring support without requiring a separate toolchain. Cascade's DSL provides two complementary syntax tiers that match ceremony to routing complexity (Figure 3). For simple, fire-and-forget messaging, a *property-based* syntax auto-executes immediately (e.g., `relay.To.Children.Send("Hello")`). For complex routing

requiring spatial filtering, a *fluent chain* builds a message incrementally and dispatches on an explicit `Execute()` call (e.g., `relay.Send("Hello").ToDescendants().Active().Within(10f).Execute()`). Each method in the chain returns a builder object, enabling IDE autocompletion and compile-time validation.

Beyond hierarchy and spatial filters, the DSL supports type filtering (`OfType<T>`, `WithComponent<T>`) for targeting specific responder types, predicate filtering (`Where`, `OnLayer`) for arbitrary runtime conditions, and reactive listener subscriptions for event-driven workflows. These capabilities compose with the spatial and hierarchy filters described below.

3.3 Spatial Filtering

Interactive applications frequently need to deliver messages based on spatial relationships. For example, an explosion effect should notify only nearby objects, a robot fault alert should reach only workstations within a safety zone, and a directional sound should reach objects within an audio cone. Without native support, developers resort to one of three patterns: setting up physics trigger volumes and wiring callbacks, collecting spatial targets and dispatching to each individually, or writing manual distance-check loops. All three separate the spatial predicate from the message dispatch, forcing the developer to manage both. Figure 4 compares three approaches for a representative task: sending a proximity alert to all active objects within 10 meters.

Cascade's spatial filters encapsulate distance and direction logic through four primitives:

- `Within(radius)`: Euclidean distance from sender.
- `InCone(direction, angle, distance)`: Conical region.
- `InDirection(direction, maxAngle)`: Directional hemisphere.
- `InBounds(bounds)`: Axis-aligned bounding box.

Spatial filters compose with hierarchy filters, enabling queries such as "notify all active descendants within 10 meters." Across our three evaluation tasks, Cascade requires 7–8 LOC (one dispatch call plus a fixed receiver pattern) compared to 15–21 LOC for Unity Events (Figure 4).

3.4 Standard Library

Cascade includes a standard library of 16 pre-built message types (8 UI, 8 input) covering common interaction patterns with relevant metadata, reducing boilerplate and establishing shared conventions. For example, `MmInput6DOFMessage` carries handedness, position, rotation, velocity, and tracking status for an XR controller, while `MmUIClickMessage` carries screen position, click count, and button index.

3.5 Static Analysis Support

Prior work suggests that integrated tool support can significantly improve code correctness [24, 40]. Cascade includes 15 Roslyn analyzers (compile-time code inspections built on the .NET Compiler Platform) that detect common messaging errors across three categories: structural warnings, API guidance, and advanced diagnostics. Two include code fix providers offering one-click resolution (e.g., omitting the terminal `Execute()` call triggers an IDE error with a quick-fix to insert it).

```
foreach (var obj in allTargets) {
  if (!obj.activeInHierarchy) continue;
  float dist = Vector3.Distance(transform.position, obj.transform.
      position);
  if (dist <= 10f) obj.GetComponent<IAlertHandler>()?.OnAlert(
      alertData);
}
```

(a) Unity Events: 12 lines, 2 files.

```
// --- Installer.cs ---
builder.RegisterMessageBroker<AlertMessage>(MessagePipeOptions.
      Default);
// --- Publisher.cs ---
[Inject] IPublisher<AlertMessage> pub;
void SendAlert(AlertData d) { pub.Publish(new AlertMessage(d)); }
// --- Subscriber.cs ---
[Inject] ISubscriber<AlertMessage> sub;
IDisposable subscription;
void Start() {
  subscription = sub.Subscribe(msg => {
    float dist = Vector3.Distance(transform.position, msg.Origin);
    if (dist <= 10f && gameObject.activeInHierarchy) HandleAlert(
      msg);
  });
}
void OnDestroy() => subscription?.Dispose();
```

(b) MessagePipe: 21 lines, 3 files.

```
relay.Send(alertData).ToDescendants().Active().Within(10f).Execute();
```

(c) Cascade: 1 expression, 1 file.

Figure 4: Three approaches to proximity-filtered messaging (alert active objects within 10 m). Cascade (c) replaces 12–21 lines with one expression, requiring no explicit subscription, unlike (a–b): receivers inherit routing from scene hierarchy and handle messages via method overrides (Section 3.2).

3.6 Performance Characteristics

Cascade achieves high performance through three mechanisms: (1) *object pooling* of message objects, predicate lists, and cycle-detection sets to reduce garbage collection pressure (the fluent builder chain is struct-based and allocates no heap memory); (2) *precomputed routing tables* that enable direct iteration over eligible responders without runtime discovery (`GetComponentsInChildren` or reflection); and (3) *lazy spatial evaluation* that skips distance calculations when hierarchy filters already exclude all candidates.

At a representative scale ($N = 8$ receivers), `Within()` completes in 1.19 µs, consuming 0.011% of a 90 fps frame budget (11.1 ms). Detailed scaling analysis is reported in Section 4.

3.7 Unified Local/Network Architecture

Cascade provides a unified messaging architecture where the same fluent syntax routes messages through local scene-graph hierarchies or across networked peers. Network behavior is controlled through filter methods: `.OverNetwork()` for local-and-network delivery, or `.NetworkOnly()` to skip local delivery. The architecture abstracts network backends through the `IMmNetworkBackend` interface, with implementations for FishNet and Photon Fusion 2. Spatial

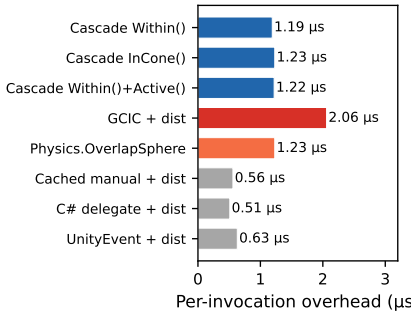


Figure 5: Per-invocation overhead for spatial fan-out at $N = 8$, $D = 1$. Cascade matches Physics.OverlapSphere and is 1.7× faster than the uncached GCIC pattern.

filtering is evaluated locally before network transmission: a message with `Within(10f)` filters recipients on the sender's machine, then transmits only to those that passed the spatial predicate.

3.8 Message Propagation in Asymmetric Graphs

Real-world networked applications frequently exhibit graph asymmetry: late-joining participants encounter evolved scenes, role-based experiences present different UIs, and HRI scenarios may maintain fundamentally different scene structures for operators and robots [9]. We analyzed Cascade's behavior under three asymmetry scenarios: a message targets a node that exists (1) locally but not remotely or (2) remotely but not locally; and (3) hierarchies diverge such that the same logical node has different ancestry paths.

Cascade resolves node identity through native networking framework identifiers for spawned objects and deterministic path-based hashing for scene objects. When a target node cannot be resolved, the message is logged and dropped. XR messaging is soft real-time: a missed proximity alert is preferable to a propagation failure that stalls the rendering loop, and the next dispatch cycle will deliver current state to any nodes that have since become reachable. The hierarchical structure provides partial robustness: messages targeting descendants still reach nodes that exist on both peers, even if sibling branches differ, contrasting with flat pub/sub systems where topic-based routing provides no structural fallback.

4 System Performance

We benchmarked Cascade's spatial filtering primitives (C1) against eight baselines representing real Unity development patterns. All baselines read live `transform.position` (no pre-cached vectors). Benchmarks ran on a desktop (i9-12900K, 32 GB, RTX 4090, Windows 11, Unity 6.3 LTS) as IL2CPP standalone builds, with 2-second measurement windows, 3 runs per configuration (run 0 warmup excluded), and Fisher–Yates shuffled method order. Objects were placed randomly in a 40 × 40 m area with a 5 m spatial radius (~5% match rate). We report per-invocation overhead in µs relative to a 90 fps frame budget (11.1 ms), where N = receiver count and D = hierarchy depth.

Small-scale performance ($N = 8$, $D = 1$). Figure 5 and Table 1 compare all approaches at a representative small scale. Cascade's `Within()` completes in 1.19 µs (0.011% of the frame budget), matching

Table 1: Per-invocation overhead (µs) for spatial fan-out at $D = 1$. Cascade primitives (top) vs. baselines (bottom). All methods read live `transform.position`.

Method	Per-invocation overhead (µs)				
	$N=4$	$N=8$	$N=16$	$N=32$	$N=64$
Cascade `Within()`	1.08	1.19	1.40	2.46	3.23
Cascade `InCone()`	1.18	1.23	1.58	2.08	3.10
Cascade `Within()+Active()`	1.09	1.22	1.53	2.42	3.57
Uncached GCIC + dist.	1.84	2.06	2.58	4.12	5.99
Physics OverlapSphere	1.19	1.23	1.18	2.58	2.75
Cached manual + dist.	0.43	0.56	0.76	1.35	2.14
C# delegate + dist.	0.39	0.51	0.72	1.34	2.19
UnityEvent + dist.	0.52	0.63	0.83	1.37	2.48

Unity's native `Physics.OverlapSphereNonAlloc` (1.23 µs) without requiring collider components. Cascade is 1.7× faster than the uncached `GetComponentsInChildren` (GCIC) + distance loop pattern (2.06 µs), a common Unity development approach. Cached manual iteration (0.56 µs) and C# delegates with subscriber-side filtering (0.51 µs) are ~2× faster, representing the expected cost of Cascade's routing abstraction (~0.5–1 µs fixed overhead for message pooling and metadata, plus ~50–80 ns per matched target for virtual dispatch).

Width scaling. As N increases at $D = 1$ (Figure 6), Cascade's spatial fan-out cost grows from 1.19 µs ($N = 8$) to 3.23 µs ($N = 64$), remaining under 0.03% of frame budget. The advantage over uncached GCIC grows from 1.7× at $N = 8$ to 1.9× at $N = 64$, because GCIC's $O(hierarchy)$ reflection cost scales worse than Cascade's pre-indexed iteration. Physics pulls ahead at $N = 64$ (2.75 µs vs. 3.23 µs, 1.2×) due to its native C++ bounding volume hierarchy with single instruction, multiple data acceleration.

Depth scaling. At $N = 64$, increasing hierarchy depth from $D = 1$ to $D = 3$ (branching factor 4) raises Cascade's `Within()` cost from 3.23 to 4.96 µs (+53%), reflecting per-level traversal overhead, but it still outperforms uncached GCIC at $D = 3$ (4.96 vs. 5.62 µs, 1.1×).

Comparison with Unity Physics. Cascade's pure-C# spatial filtering matches `Physics.OverlapSphereNonAlloc` at $N = 8$ (1.19 vs. 1.23 µs) without requiring collider components on target objects. At larger N, Physics' native bounding volume hierarchy pulls ahead (2.75 vs. 3.23 µs at $N = 64$). However, physics queries cannot compose spatial predicates with hierarchy direction, message type, or active-state filters in a single dispatch call; Cascade integrates spatial filtering directly into its routing infrastructure.

Memory. The fluent builder chain is struct-based and allocates no heap memory. Residual per-dispatch allocations are bounded and do not grow with sustained load (>1000 msg/s for 60 s).

5 User Study

We conducted a within-subjects experiment comparing Cascade with traditional Unity Events to evaluate our DSL's impact on developer productivity and experience.

5.1 Research Questions and Hypotheses

We organize the evaluation around five hypotheses spanning three research questions that map to our contributions.

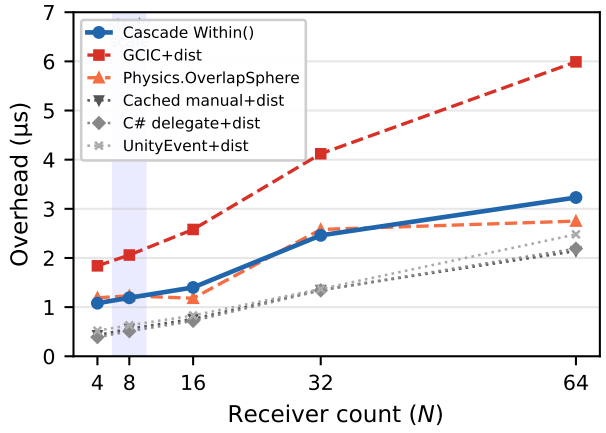


Figure 6: Width scaling ($D = 1$). Cascade `Within()` (blue) tracks between Physics.OverlapSphere and the uncached GCIC pattern across $N = 4$–64. Shaded region: $N = 8$.

RQ1: *Does integrating spatial predicates into message routing reduce developer effort?* (C1)

- **H1 (Correctness)**: *Cascade users achieve higher code correctness scores than Unity Events users.* Because tasks are time-boxed, correctness at the deadline is the primary behavioral measure, following methodology established in prior DSL [36] and software engineering experiments [22] that use partial correctness under fixed time as the primary dependent variable. We additionally report compile error count and lines of code as secondary indicators of code quality and API conciseness.
- **H2 (Task Completion)**: *Cascade users achieve higher task completion rates than Unity Events users.*

RQ2: *Does our DSL reduce cognitive load and improve usability?* (C2)

- **H3 (Workload)**: *Cascade users report lower overall cognitive load (NASA-TLX) than Unity Events users.*
- **H4 (Usability)**:*Cascade users report higher system usability (SUS) than Unity Events users.* We additionally report preference ranking.

RQ3: *Does the DSL produce a more focused development workflow?* (C1+C2)

- **H5 (Workflow focus)**: Cascade users exhibit lower gaze-based AOI transition entropy than Unity Events users, computed from fixation transitions across code editor, reference documentation, and Unity Editor panels (Game View, Inspector, Hierarchy, Console) via Tobii Pro Fusion at 120 Hz. We additionally report application-level dwell proportions from window focus logs as a secondary indicator of workflow structure.

Each confirmatory hypothesis (H1–H5) tests a distinct construct (correctness, task completion, workload, usability, workflow) and is evaluated independently at $\alpha = 0.05$.

5.2 Participants

We recruited 14 participants (8 men, 6 women; $\bar{x}_{\text{age}} = 23.4$, range 18–34) from university computer science and engineering programs. All held or were pursuing CS-related degrees (7 BS in progress, 5 MS in progress, 1 MS completed, 1 PhD in progress). Self-reported programming experience ranged from 1–3 years ($n = 4$) to 5–10 years ($n = 2$), with the majority at 3–5 years ($n = 8$). Unity experience was 3 months to 1 year ($n = 9$) or 1–3 years ($n = 5$); mean self-rated C# proficiency was 3.6/7 and Unity proficiency 3.4/7. Inclusion criteria required at least 3 months of Unity experience, completion of at least one Unity project, and normal or corrected-to-normal vision (required for eye tracking). Exclusion criteria, defined a priori, were: prior experience with the Cascade framework and failure to complete training exercises. Participants received modest compensation (suppressed for review) for the approximately 90-minute session. This sample size is consistent with prior developer tool evaluations, which typically report $n = 10$–20 for within-subjects designs [13, 65, 71].

5.3 Power Analysis

Based on effect sizes from prior DSL evaluations ($d = 0.51$–0.81) [36], we conservatively assumed $d = 0.80$. A simulation-based power analysis using the `simr` package for R [25] with 1,000 Monte Carlo replications confirmed that $n = 14$ provides $\geq 97\%$ power for the primary LMM analysis (H1), which benefits from repeated measures (3 tasks $\times$ 2 conditions = 6 observations per participant), and $\geq 89\%$ power for paired tests (H3–H5). Full simulation parameters are provided in Appendix I.

5.4 Conditions

Participants completed tasks under two conditions in counterbalanced order. We selected Unity Events as the baseline because they are the most widely used messaging approach in Unity [67], all participants had at least 3 months of prior familiarity, and the API ships with the engine. We excluded third-party frameworks (e.g., MessagePipe [10]) because their dependency injection containers confound any messaging comparison; Figure 4 provides a code comparison with MessagePipe for reference. Each condition included 12 minutes of training (syntax, spatial/structural patterns, practice exercise). This training asymmetry (12 minutes vs. 3 months of prior Unity experience) is a conservative design choice: any results favorable to Cascade obtained under this disadvantage represent a lower bound on its benefits with equal exposure.

5.5 Tasks

Participants completed three construction tasks under each condition, spanning a robot teleoperation workspace (T1), a factory IoT monitoring environment (T2), and a command & control scenario (T3). Each task had its own Unity scene; tasks were time-boxed at 8 minutes each. The three tasks cover complementary communication patterns: T1 (Safety Zone Alerts) tests spatial filtering (C1), T2 (Alert Aggregation) tests bidirectional hierarchy routing (C2), and T3 (Drone Swarm Command) tests broadcast-down plus notify-up with ACK forwarding. This cross-domain design tests whether Cascade's benefits generalize beyond a single application domain.

For all tasks, scenes contained visual GameObjects with shared infrastructure components (indicators, drone units, subsystem nodes) pre-attached. For Cascade, relay node components were present on all relevant GameObjects but routing tables were empty; participants wired routing tables (in the Inspector or via code) and

wrote task logic. Participants in both conditions could use Inspector-based or code-based approaches for component wiring; reference materials presented all available options.

Scenes were partially scaffolded with starter scripts containing goal descriptions and TODO comments but no messaging logic. Starter scripts omitted exact API syntax; participants consulted a browser-based reference card for method signatures, switching between VS Code, Unity Editor, and the reference browser via Alt-Tab. All tasks used VS Code with a standardized profile (18 pt font, minimap disabled, layout-altering shortcuts disabled) to ensure consistent IDE conditions (Figure 7).

T1: Safety Zone Alerts (Spatial Filtering, HRI). Eight safety zone indicators surround a robot arm at varying distances. Participants implemented proximity-based alerting with three distance thresholds (1 m danger, 2 m warning, 3.5 m caution), inspired by ISO/TS 15066 speed and separation monitoring [8, 58]. Indicators within each threshold receive the corresponding alert level; those beyond 3.5 m automatically reset to a clear state after 0.5 s without alerts (no explicit clear message required). We excluded Unity's physics API (trigger volumes, `OverlapSphere`) because the tasks require on-demand message dispatch rather than continuous proximity detection; using physics would test engine knowledge rather than messaging patterns. For Cascade, participants wrote three `relay.Send().ToDescendants().Within()` calls with different radii. For Events, participants used `GetComponentsInChildren` from the parent transform to find all sibling indicators dynamically, then iterated with `Vector3.Distance` checks. Because Cascade's three `Within()` calls are not mutually exclusive (an indicator at 0.8 m receives all three alert levels), the `SafetyZoneIndicator` responder implements priority logic that retains only the highest-severity alert per frame. The Events condition uses sender-side if/else-if branching, which is inherently exclusive. Both approaches produce identical indicator states; the difference lies in where exclusion is enforced (receiver vs. sender).

T2: Alert Aggregation (Bidirectional Hierarchy, IoT). Four independent IoT subsystems (HVAC, Electrical, Hydraulics, and Cooling) each publish status alerts upward to a central dashboard, which in turn sends diagnostic check requests back down to all subsystems. This bidirectional pattern tests how each framework handles simultaneous upward aggregation and downward command dispatch through the same components. For Cascade, each subsystem uses a single relay node for both directions. For Events, participants wired separate upward (C# events) and downward (`UnityEvent` listeners) paths with explicit component discovery.

T3: Drone Swarm Command (Broadcast + ACK, Command & Control). A drone swarm scenario requires implementing a mission command method on a central command station: broadcast a mission order to six surrounding drone units, report dispatch upward to a monitoring display, and collect per-drone acknowledgments back up through the same relay. This task tests two complementary routing directions: broadcast down to all drones and notify-up for aggregation, plus the reuse of a single script for both outgoing commands and incoming ACKs. Notably, T3's routing graph differs from the Unity Transform hierarchy: the command station serves as a routing parent for all drone units even though both are Transform children of a shared grouping node, demonstrating that Cascade's routing topology is independent of the scene graph. Drone units are pre-built infrastructure; participants write only the `CommandStation` script (one script per condition). For Cascade, participants wrote two dispatch calls and one responder override. For Events, participants discovered drones via `GetComponentsInChildren`, dispatched commands in a foreach loop, and subscribed to per-drone acknowledgment events.

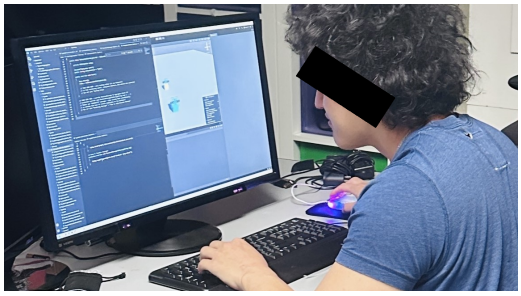


Figure 7: Study apparatus: 24" 1920×1080 monitor with Tobii Pro Fusion eye tracker, full-screen Unity Editor with fixed panel layout, and window for VS Code or reference card.

5.6 Measures

We collected behavioral, workflow, and subjective measures. Eye tracking data was recorded using a Tobii Pro Fusion (120 Hz) via Tobii Pro Lab. Application-level transitions (VS Code, Unity Editor, reference card browser) were captured via a background window focus logger; within the Unity Editor, four panel-level areas of interest (AOIs) were defined for the Game View, Inspector, Hierarchy, and Console panels. All task phases were screen-recorded for post-hoc behavioral analysis. The study protocol was approved by our institutional review board.

Behavioral Measures (H1, H2):

- Task correctness: rubric-based partial score normalized to $[0, 1]$ per task (H1; see Appendix B). Following Fucci et al. [22], who operationalize functional correctness as the percentage of acceptance tests passed within a fixed task duration, and Johanson and Hasselbring [36], who measure correctness as the fraction of correctly implemented features, we use partial correctness at the 8-minute deadline as the primary behavioral measure.
- Task completion rate: binary pass/fail (H2).
- Compile error count from Unity Console logs (H1, secondary).
- Lines of code written (H1, secondary).

Subjective Measures (H3, H4):

- NASA-TLX [28] administered per condition (H3).
- System Usability Scale (SUS) [5] administered per condition (H4).
- Preference ranking (H4, secondary).
- Open-ended feedback.

Workflow Measures (H5):

- Gaze-based AOI transition entropy: Shannon entropy of the fixation transition matrix across code editor, reference documentation, and Unity Editor panels (Game View, Inspector, Hierarchy, Console), from Tobii Pro Fusion at 120 Hz (H5).

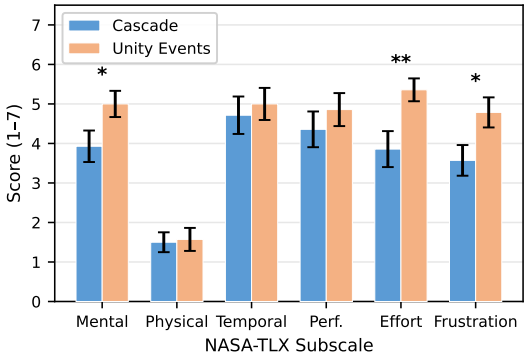


Figure 8: NASA-TLX subscale scores by condition (mean ± SE). Lower scores indicate lower perceived workload. Cascade participants reported significantly lower Mental Demand, Effort, and Frustration. $^{*}p < .05$; $^{**}p < .01$ (Wilcoxon signed-rank, two-tailed).

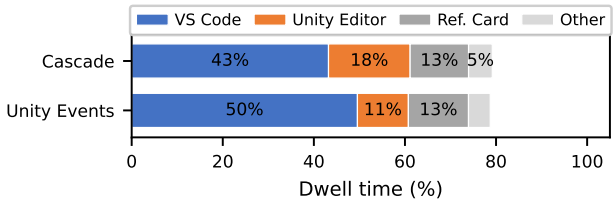


Figure 9: Application dwell time proportions by condition. Cascade participants spent more time in the Unity Editor, reflecting routing table configuration.

- Application-level dwell proportions: percentage of task time per application (VS Code, Unity Editor, reference browser), from window focus logs at 2 Hz (H5, secondary).

5.7 Procedure

Each approximately 90-minute session began with consent, demographics, and eye tracker calibration, followed by two condition blocks separated by a break. Within each block, participants received 12 minutes of training, completed a calibration drift check, worked on all three tasks (T1–T3, each time-boxed at 8 minutes), and completed NASA-TLX and SUS questionnaires. A preference ranking was collected at the end of the second block. Condition order was counterbalanced (AB/BA), and task order was counterbalanced using a balanced Latin square (3 orderings) crossed with condition order, yielding 6 groups with 2–3 participants per group (Appendix E). Each participant received the same task order under both conditions to ensure that condition differences could not be attributed to differential task sequencing. Screen, audio, and eye tracking data (pupil diameter and gaze position at 120 Hz) were recorded throughout all task phases. A pilot study with 3 participants (not in the final sample) confirmed that 8-minute time boxes produced meaningful variation in partial correctness.

6 Results

6.1 Analysis Approach

Our primary analysis uses linear mixed-effects models (LMMs) with condition (Cascade vs. Unity Events) and task (T1–T3) as fixed effects, their interaction, and participant as a random intercept. We fit a maximal random effects structure (random slopes for condition and task by participant) following Barr et al. [2]; if models fail to converge, we simplify by removing the correlation term, then random slopes for task, then random slopes for condition, reporting the final structure. The condition × task interaction tests whether Cascade's advantage varies across communication patterns and is treated as exploratory. We include self-reported Unity experience (mean-centered), condition presentation order (AB vs. BA), and task presentation order (Latin square sequence) as covariates.

H1 (correctness) is evaluated using the LMM main effect of condition on partial correctness scores. For binary outcomes (task completion, H2), we use generalized linear mixed-effects models with a logit link. For ordinal data (NASA-TLX subscales), we use Wilcoxon signed-rank tests. H3 (NASA-TLX) and H4 (SUS) use Wilcoxon signed-rank tests on per-condition scores. H5 (workflow focus) uses a paired t-test on per-participant gaze-based AOI transition entropy.

Tasks are time-boxed at 8 minutes; partial correctness at the deadline serves as the primary behavioral measure (H1), following fixed-time designs in software engineering [22, 36]. Fucci et al. [22] measured the "percentage of acceptance asserts passed [...] by the production code delivered by a participant within the fixed duration of the task"; Johanson and Hasselbring [36] measured correctness as the fraction of correctly implemented features. Models are fit with REML; nested comparisons use ML via likelihood ratio tests. We check residual normality and homoscedasticity for all LMMs; because correctness scores are bounded $[0, 1]$, we additionally fit beta regression as a sensitivity analysis. We report 95% confidence intervals and Cohen's d for all comparisons. Our five confirmatory hypotheses group into two families: *task performance* (H1 correctness, H2 completion) and *subjective experience* (H3 workload, H4 usability); H5 (workflow) stands alone. We apply Holm-Bonferroni correction [32] within each family ($k = 2$), testing the smaller p at $\alpha/2 = 0.025$ and the larger at $\alpha = 0.05$; H5 is evaluated independently at $\alpha = 0.05$. The condition × task interaction, per-task breakdowns, and T3 composability (whether benefits of C1 and C2 compound) are exploratory with uncorrected p-values.

6.2 Task Performance

The maximal LMM (random slopes for condition and task by participant) converged; we report this model throughout. The LMM revealed a significant main effect of condition on partial correctness ($\beta = 0.338$, $SE = 0.050$, $t = 6.72$, $p < .001$, $d = 1.47$), supporting H1: Cascade participants achieved higher correctness scores than Unity Events participants across all three tasks. Per-task paired comparisons were significant for all tasks: T1 ($d = 0.84$, $p = .008$), T2 ($d = 1.75$, $p < .001$), and T3 ($d = 2.79$, $p < .001$), with the largest effect for T3 (Drone Swarm Command), where composing multiple routing primitives amplified Cascade's advantage. Correctness scores (rubric 0–1) were higher for Cascade across all tasks (T1: 0.84 vs. 0.64; T2: 0.76 vs. 0.44; T3: 0.77 vs. 0.28). Of 42 task instances

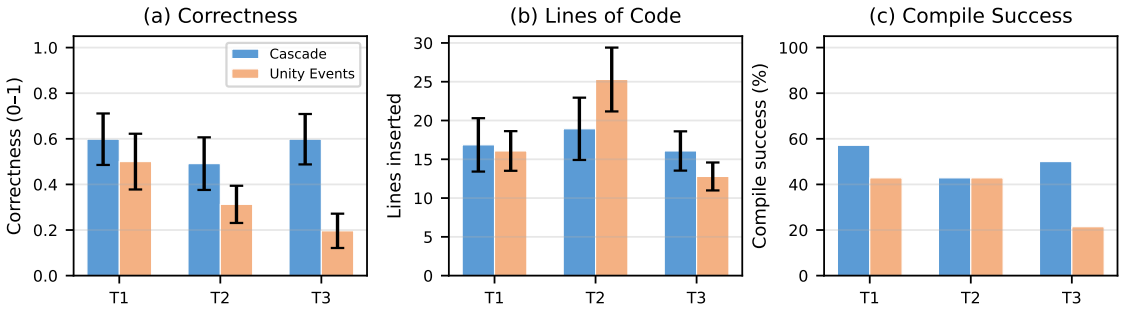


Figure 10: Task performance: Cascade (blue) vs. Unity Events (orange) for (a) correctness, (b) lines of code, and (c) compile success rate across T1–T3. All tasks were time-boxed at 8 minutes. Error bars show ±1 SE.

per condition, 12 Cascade submissions achieved full correctness (score = 1.0) compared to 8 for Events; however, per-participant completion rates did not differ significantly ($W = 9.0$, $p = .196$, $d = 0.32$), so H2 is not supported. All tasks were time-boxed at 8 minutes; no participant finished all three tasks under either condition, confirming that the time constraint produced meaningful variation in partial scores. Figure 10 summarizes these results.

6.3 Cognitive Load

Cascade participants reported significantly lower overall NASA-TLX workload than Unity Events participants ($W = 9.0$, $p = .011$, $r = .68$, $d = -0.89$), supporting H3. Figure 8 shows subscale scores by condition. The largest subscale differences were on Effort (Cascade: $\tilde{x} = 4$; Events: $\tilde{x} = 6$; $W = 11.0$, $p = .026$) and Frustration (Cascade: $\tilde{x} = 4$; Events: $\tilde{x} = 4$; $W = 21.0$, $p = .011$), followed by Mental Demand (Cascade: $\tilde{x} = 4$; Events: $\tilde{x} = 5$; $W = 10.5$, $p = .018$). Temporal Demand was marginally lower for Cascade ($p = .058$). Physical Demand subscales did not differ significantly (both conditions $\tilde{x} \leq 1.5$), as expected for a seated coding task.

6.4 Attention Allocation

We analyzed attention allocation using two complementary data sources: application-level dwell time from window-switch logs (2 Hz) and gaze-based AOI dwell from the Tobii Pro Fusion (120 Hz, $n = 42$ task observations). Figure 9 shows dwell time proportions. At the application level, participants in both conditions spent similar time in the code editor (Cascade 43.2% vs. Events 49.6%) and reference documentation (Cascade 12.8% vs. Events 13.2%). Cascade participants spent more time in the Unity Editor overall (Cascade 17.9% vs. Events 11.2%), reflecting the routing table setup performed in the Inspector. Gaze-based AOI analysis revealed that Events participants spent proportionally more time on the reference card (Events 41.3% vs. Cascade 36.6%), while Cascade participants spent more time viewing the Unity Console panel (Cascade 11.8% vs. Events 6.0%), consistent with iterative compile-and-test cycles during routing configuration. Inspector panel dwell was low in both conditions (Cascade 1.9%, Events 1.4%). Gaze-based AOI transition entropy showed a small non-significant difference ($t(13) = 1.44$, $p = .174$, $d = 0.38$), with Cascade participants exhibiting slightly higher entropy ($\bar{x} = 2.06$) than Events participants ($\bar{x} = 1.93$), suggesting marginally more distributed attention across panels. H5 is not supported: both conditions produced similar workflow structure.

6.5 Usability

Cascade received significantly higher SUS scores than Unity Events (Cascade: $\tilde{x} = 60.0$, IQR = $[45.6, 71.2]$; Events: $\tilde{x} = 48.8$, IQR = $[28.1, 52.5]$; $W = 82.0$, $p = .032$, $r = .50$, $d = 0.58$), supporting H4 (Figure 11). 11 of 14 participants scored Cascade higher than Unity Events on SUS. 9 of 14 participants preferred Cascade overall; 3 preferred Events, citing familiarity with code-based wiring and fewer hierarchy dependencies; 2 expressed no preference. When asked which approach they would use in a new project, 8 chose Cascade, 4 chose Events, and 2 said "it depends."

Paired self-report comparisons (7-point Likert) favored Cascade across all five dimensions, with the largest difference for proximity-based messaging ($d = 0.88$), followed by hierarchy wiring reduction ($d = 0.66$), code predictability ($d = 0.65$), learnability ($d = 0.64$), and debugging ease ($d = 0.60$). None reached significance at $n = 14$, consistent with the SUS effect size ($d = 0.58$) suggesting that subjective preference measures require larger samples than behavioral measures for this paradigm comparison.

All three supported hypotheses (H1, H3, H4) remain significant under Holm-Bonferroni correction within their respective families ($k = 2$; H1 $p < .001 < .025$, H3 $p = .011 < .025$, H4 $p = .032 < .05$).

6.6 Qualitative Feedback

Open-ended responses were coded inductively using thematic analysis [4]. Three themes emerged.

Conciseness and unified API: Multiple participants highlighted the reduced code volume as a primary benefit. One noted "it was a lot less code to write compared to [Events]"; another described Cascade's syntax as "intuitive and easy to code…I could accomplish the same thing with less code." A third summarized that Cascade was "overall less demanding than [Events] by a significant margin."

Meaningful abstractions for spatial and hierarchical messaging: Participants valued the named operations that mapped directly to their intent. One participant wrote that "the BroadcastValue(), NotifyValue(), and Within() functions made a great deal of sense"; another described the benefit as "simple…to broadcast down or notify up" and "link once and done." In contrast, Events participants

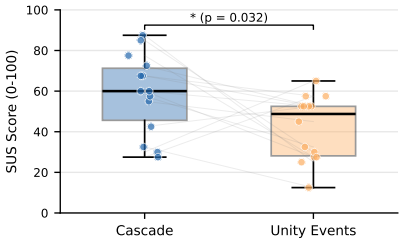


Figure 11: System Usability Scale (SUS) scores by condition. Box plots show median and IQR; dots are individual participants; lines connect paired observations. $^{*}\,p < .05$ (Wilcoxon signed-rank, two-tailed).

noted that "you had to link each action with a specific function and it could get very tedious."

Learning curve for hierarchy setup: Several participants reported initial confusion about configuring parent–child relationships and routing tables. One participant wrote, "the most difficult part was learning how to connect the parent and children"; another described "lots of nested parent child layering that makes it difficult to figure out what to use and where." Participants who preferred Events cited "more familiar workflow without hierarchy dependency concerns" and having "more coding experience than Unity [Inspector experience]."

7 Discussion

Cascade's fluent DSL produced consistent advantages over Unity Events across correctness, cognitive load, and usability, addressing C3's goal of providing the first empirical evaluation of spatially-aware message routing. The largest correctness effect occurred for T3 ($d = 2.79$), where the composable DSL replaced per-drone iteration, event subscription management, and separate ACK handlers with a single `BroadcastValue` call and automatic upward propagation. T2 ($d = 1.75$) and T1 ($d = 0.84$) showed large effects, with the strongest gains for tasks requiring composition of multiple routing primitives. The lower cognitive load (H3) and higher usability (H4) indicate that the fluent DSL reduces both objective effort and subjective burden. Task completion rates (H2) and gaze transition entropy (H5) did not differ significantly, suggesting that Cascade's advantages manifest in code quality and perceived workload rather than completion speed or workflow structure.

7.1 Implications for Interactive Applications

Our results suggest three implications for messaging API design. First, spatial awareness should be a first-class concern: while spatial predicates are established at the network level [18, 62], integrating them into intra-application dispatch yields measurable productivity gains. Second, fluent interfaces reduce viscosity and improve mapping closeness [26] when combined with IDE autocompletion. Each method returns a typed builder whose members are the valid next steps, so the autocompletion menu doubles as a reference card. Third, compile-time error detection may reduce debugging effort, a significant barrier for end-user programmers [39].

7.2 Cross-Domain Applicability

Our cross-domain task design demonstrates that Cascade's patterns generalize: one participant noted it was "overall less demanding than [Events] by a significant margin" across HRI and IoT tasks. The growing use of game engines for HRI [9, 61] and XR-based IoT [45, 55] creates demand for principled intra-application communication, and Cascade's hierarchical routing maps naturally to multi-tier architectures [1] and asymmetric peer topologies [59].

7.3 Limitations and Future Work

The 90-minute session limited task complexity; real-world projects may show larger benefits. We compared against Unity Events, a deliberately strong baseline (Section 5.4); comparison with MessagePipe [10] or reactive approaches [11] would clarify when hierarchical semantics provide value over flat pub/sub. The 12-minute training asymmetry (vs. three months to two years of Unity experience) is conservative; we include Unity experience as a covariate. Future work includes a visual debugging tool for real-time message flow visualization [30], longitudinal studies of code maintainability, explicit asymmetry handling for divergent HRI scene graphs [9], and extension beyond Unity to other scene-graph frameworks.

8 Conclusions

We presented Cascade, a fluent DSL for hierarchical message routing in scene-graph–based applications that integrates composable spatial filtering primitives into a self-documenting routing API. Our controlled experiment with 14 developers across robot teleoperation and IoT monitoring tasks demonstrates that Cascade improves code correctness ($d = 1.47$), reduces cognitive load ($d = 0.89$), and improves usability ($d = 0.58$) compared to event-based messaging. The largest benefits occurred for multi-pattern composition tasks ($d = 2.79$), validating the design decision to integrate spatial, hierarchical, and broadcast primitives into a unified composable API. Cascade is available as open source.

References

[1] Shutaro Aoyama, Jen-Shuo Liu, Portia Wang, Shreeya Jain, Xuezhen Wang, Jingxi Xu, Shuran Song, Barbara Tversky, and Steven K. Feiner. 2024. Asynchronously Assigning, Monitoring, and Managing Assembly Goals in Virtual Reality for High-Level Robot Teleoperation. In *Proceedings of the 2024 IEEE Conference on Virtual Reality and 3D User Interfaces (IEEE VR '24)*. IEEE. doi:10.1109/VR58804.2024.00066

[2] Dale J. Barr, Roger Levy, Christoph Scheepers, and Harry J. Tily. 2013. Random Effects Structure for Confirmatory Hypothesis Testing: Keep It Maximal. *Journal of Memory and Language* 68, 3 (2013), 255–278. doi:10.1016/j.jml.2012.11.001

[3] Steve Benford and Lennart Fahlén. 1993. A Spatial Model of Interaction in Large Virtual Environments. In *Proceedings of the Third European Conference on Computer-Supported Cooperative Work (ECSCW '93)*. Springer, Milan, Italy, 109–124. doi:10.1007/978-94-011-2094-4_8

[4] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[5] John Brooke. 1996. SUS: A 'Quick and Dirty' Usability Scale. In *Usability Evaluation in Industry*. Taylor & Francis, 189–194.

[6] Paulo Canelas, Miguel Tavares, Ricardo Cordeiro, Alcides Fonseca, and Christopher S. Timperley. 2022. An Experience Report on Challenges in Learning the Robot Operating System. In *Proceedings of the 4th International Workshop on Robotics Software Engineering (RoSE '22)*. ACM, 1–8. doi:10.1145/3526071.3527521

[7] Yuanzhi Cao, Tianyi Wang, Xun Qian, Pawan S. Rao, Manav Wadhawan, Ke Huo, and Karthik Ramani. 2019. GhostAR: A Time-space Editor for Embodied Authoring of Human-Robot Collaborative Task with Augmented Reality. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. ACM, 521–534. doi:10.1145/3332165.3347902

[8] Yunus Cogurcu and Steve Maddock. 2023. Augmented Reality Safety Zone Configurations in Human-Robot Collaboration: A User Study. In *Companion of the 2023 ACM/IEEE International Conference on Human-Robot Interaction (HRI '23 Companion)*. ACM. doi:10.1145/3568294.3580106

[9] Enrique Coronado, Shunki Itadera, and Ixchel G. Ramirez-Alpizar. 2023. Integrating Virtual, Mixed, and Augmented Reality to Human–Robot Interaction Applications Using Game Engines: A Brief Review of Accessible Software Tools and Frameworks. *Applied Sciences* 13, 3 (2023), 1292. doi:10.3390/app13031292

[10] Cysharp. 2021. MessagePipe: High-Performance in-memory/distributed messaging pipeline for .NET and Unity. https://github.com/Cysharp/MessagePipe Accessed: 2026-01-31.

[11] Cysharp. 2024. R3: A New Modern Reimplementation of Reactive Extensions for C#. https://github.com/Cysharp/R3 Accessed: 2026-01-31.

[12] Mustafa Doga Dogan, Eric J. Gonzalez, Karan Ahuja, Ruofei Du, Andrea Colaço, Johnny Lee, Mar González-Franco, and David Kim. 2024. Augmented Object Intelligence with XR-Objects. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. doi:10.1145/3654777.3676379

[13] Brian Ellis, Jeffrey Stylos, and Brad Myers. 2007. The Factory Pattern in API Design: A Usability Evaluation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*. IEEE, 302–312. doi:10.1109/ICSE.2007.85

[14] Carmine Elvezio. 2021. *XR Development with the Relay and Responder Pattern*. Ph. D. Dissertation. Columbia University. https://academiccommons.columbia.edu/doi/10.7916/d8-k5ve-hk48

[15] Carmine Elvezio, Mengu Sukan, and Steven Feiner. 2018. Mercury: A Messaging Framework for Modular UI Components. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. ACM, New York, NY, USA, 1–12. doi:10.1145/3173574.3174162

[16] Franz Faul, Edgar Erdfelder, Albert-Georg Lang, and Axel Buchner. 2007. G*Power 3: A Flexible Statistical Power Analysis Program for the Social, Behavioral, and Biomedical Sciences. *Behavior Research Methods* 39, 2 (2007), 175–191. doi:10.3758/BF03193146

[17] Martin Fischbach, Dennis Wiebusch, and Marc Erich Latoschik. 2017. Semantic Entity-Component State Management Techniques to Enhance Software Quality for Multimodal VR-Systems. *IEEE Transactions on Visualization and Computer Graphics* 23, 4 (2017), 1342–1351. doi:10.1109/TVCG.2017.2657098

[18] FIWARE Foundation. 2023. FIWARE-NGSI v2 Specification. https://fiware.github.io/specifications/ngsiv2/stable/. Accessed: 2026-03-22.

[19] Martin Fowler. 2005. FluentInterface. https://martinfowler.com/bliki/FluentInterface.html Accessed: 2026-02-11.

[20] Martin Fowler. 2010. *Domain-Specific Languages*. Addison-Wesley.

[21] Sebastian Friston, Ben J. Congdon, David Swapp, Lisa Izzouzi, Klara Brandstätter, Daniel Archer, Otto Olkkonen, Felix J. Thiel, and Anthony Steed. 2021. Ubiq: A System to Build Flexible Social Virtual Reality Experiences. In *Proceedings of the 27th ACM Symposium on Virtual Reality Software and Technology (VRST '21)*. ACM. doi:10.1145/3489849.3489871

[22] Davide Fucci, Giuseppe Scanniello, Simone Romano, and Natalia Juristo. 2020. Need for Sleep: The Impact of a Night of Sleep Deprivation on Novice Developers' Performance. *IEEE Transactions on Software Engineering* 46, 1 (2020), 1–19. doi:10.1109/TSE.2018.2834900

[23] Bryan R. Galarza, Paulina Ayala, Santiago Manzano, and Marcelo V. Garcia. 2023. Virtual Reality Teleoperation System for Mobile Robot Manipulation. *Robotics* 12, 6 (2023), 163. doi:10.3390/robotics12060163

[24] Peter Leo Gorski, Luigi Lo Iacono, Dominik Wermke, Christian Stransky, Sebastian Möller, Yasemin Acar, and Sascha Fahl. 2018. Developers Deserve Security Warnings, Too: On the Effect of Integrated Security Advice on Cryptographic API Misuse. In *Proceedings of the Fourteenth Symposium on Usable Privacy and Security (SOUPS '18)*. USENIX Association, 265–281.

[25] Peter Green and Catriona J. MacLeod. 2016. SIMR: An R Package for Power Analysis of Generalized Linear Mixed Models by Simulation. *Methods in Ecology and Evolution* 7, 4 (2016), 493–498. doi:10.1111/2041-210X.12504

[26] Thomas R. G. Green and Marian Petre. 1996. Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework. *Journal of Visual Languages & Computing* 7, 2 (1996), 131–174. doi:10.1006/jvlc.1996.0009

[27] Jan Gugenheimer, Evgeny Stemasov, Julian Frommel, and Enrico Rukzio. 2017. ShareVR: Enabling Co-Located Experiences for Virtual Reality between HMD and Non-HMD Users. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (CHI '17)*. ACM, 4021–4033. doi:10.1145/3025453.3025683

[28] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. *Advances in Psychology* 52 (1988), 139–183. doi:10.1016/S0166-4115(08)62386-9

[29] Philip Heltweg, Georg-Daniel Schwarz, Dirk Riehle, and Felix Quast. 2025. An Empirical Study on the Effects of Jayvee, a Domain-Specific Language for Data Engineering, on Understanding Data Pipeline Architectures. *Software: Practice and Experience* (2025). doi:10.1002/spe.3409

[30] Valentin Heun, James Hobin, and Pattie Maes. 2013. Reality Editor: Programming Smarter Objects. In *Proceedings of the 2013 ACM Conference on Pervasive and Ubiquitous Computing Adjunct Publication (UbiComp '13 Adjunct)*. ACM, 307–310. doi:10.1145/2494091.2494185

[31] Benjamin Hoffmann, Neil Urquhart, Kevin Chalmers, and Michael Guckert. 2022. An Empirical Evaluation of a Novel Domain-Specific Language – Modelling Vehicle Routing Problems with Athos. *Empirical Software Engineering* 27, 6 (2022), 152. doi:10.1007/s10664-022-10210-w

[32] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[33] Gaoping Huang, Pawan S. Rao, Meng-Han Wu, Xun Qian, Shimon Y. Nof, Karthik Ramani, and Alexander J. Quinn. 2020. Vipo: Spatial-Visual Programming with Functions for Robot-IoT Workflows. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. ACM, 1–13. doi:10.1145/3313831.3376670

[34] Xincheng Huang, Michael Yin, Ziyi Xia, and Robert Xiao. 2024. VirtualNexus: Enhancing 360-Degree Video AR/VR Collaboration with Environment Cutouts and Virtual Replicas. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. doi:10.1145/3654777.3676377

[35] Ke Huo, Yuanzhi Cao, Sang Ho Yoon, Zhuangying Xu, Guiming Chen, and Karthik Ramani. 2018. Scenariot: Spatially Mapping Smart Things Within Augmented Reality Scenes. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (CHI '18)*. ACM, 1–12. doi:10.1145/3173574.3173793

[36] Arne N. Johanson and Wilhelm Hasselbring. 2017. Effectiveness and Efficiency of a Domain-Specific Language for High-Performance Marine Ecosystem Simulation: A Controlled Experiment. *Empirical Software Engineering* 22, 4 (2017), 2206–2236. doi:10.1007/s10664-016-9483-z

[37] Simon Julier, Marco Lanzagorta, Yohan Baillot, Lawrence Rosenblum, Steven Feiner, Tobias Höllerer, and Sabrina Sestito. 2000. Information Filtering for Mobile Augmented Reality. In *Proceedings of the IEEE and ACM International Symposium on Augmented Reality (ISAR 2000)*. IEEE, Munich, Germany, 3–11. doi:10.1109/ISAR.2000.880917

[38] Sungchul Jung, Nawam Karki, Max W. J. Slutter, and Robert Lindeman. 2021. On the Use of Multi-sensory Cues in Symmetric and Asymmetric Shared Collaborative Virtual Spaces. *Proceedings of the ACM on Human-Computer Interaction* 5, CSCW1 (2021), 1–25. doi:10.1145/3449146

[39] Amy J. Ko, Brad A. Myers, and Htet Htet Aung. 2004. Six Learning Barriers in End-User Programming Systems. In *Proceedings of the 2004 IEEE Symposium on Visual Languages - Human Centric Computing (VLHCC '04)*. IEEE, 199–206. doi:10.1109/VLHCC.2004.47

[40] Stefan Krüger et al. 2017. CogniCrypt: Supporting Developers in Using Cryptography. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE '17)*. 931–936. doi:10.1109/ASE.2017.8115707

[41] Tsung-Chi Lin, Achyuthan Unni Krishnan, and Zhi Li. 2023. The Impacts of Unreliable Autonomy in Human-Robot Collaboration on Shared and Supervisory Control for Remote Manipulation. *IEEE Robotics and Automation Letters* 8, 8 (2023), 4683–4690. doi:10.1109/LRA.2023.3287039

[42] Blair MacIntyre and Steven Feiner. 1998. A distributed 3D graphics library. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. Association for Computing Machinery, New York, NY, USA, 361–370. doi:10.1145/280814.280935

[43] Blair MacIntyre, Maribeth Gandy, Steven Dow, and Jay David Bolter. 2004. DART: A Toolkit for Rapid Design Exploration of Augmented Reality Experiences. In *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST '04)*. ACM, 197–206. doi:10.1145/1029632.1029669

[44] João Pedro Oliveira Marum, J. Adam Jones, and H. Conrad Cunningham. 2020. Dependency Graph-based Reactivity for Virtual Environments. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. IEEE, 226–233. doi:10.1109/VRW50115.2020.00052

[45] Andrea Mattioli and Fabio Paternò. 2023. A Mobile Augmented Reality App for Creating, Controlling, Recommending Automations in Smart Homes. *Proceedings of the ACM on Human-Computer Interaction* 7, ISS (2023), 1–27. doi:10.1145/3604242

[46] Luca Mehlhorn and Stefan Hanenberg. 2022. Imperative versus Declarative Collection Processing: An RCT on the Understandability of Traditional Loops versus the Stream API in Java. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, 866–877. doi:10.1145/3510003.3519016

[47] Brad A. Myers and Jeffrey Stylos. 2016. Improving API Usability. *Commun. ACM* 59, 6 (2016), 62–69. doi:10.1145/2896587

[48] Dan R. Olsen. 2007. Evaluating User Interface Systems Research. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (UIST '07)*. ACM, 251–258. doi:10.1145/1294211.1294256

[49] Open Robotics. 2025. ROS: Robot Operating System. https://www.ros.org/ Accessed: 2026-03-31.

[50] Marco Piccioni, Carlo A. Furia, and Bertrand Meyer. 2013. An Empirical Study of API Usability. In *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '13)*. IEEE, 5–14. doi:10.1109/ESEM.2013.14

[51] Thibault Raffaillac and Stéphane Huot. 2018. Applying the Entity-Component-System Model to Interaction Programming. In *Proceedings of the 30th Conference on l'Interaction Homme-Machine (IHM '18)*. ACM, 42–51. doi:10.1145/3286689.3286703

[52] Nils Reichl, Stefan Hanenberg, and Volker Gruhn. 2023. Does the Stream API Benefit from Special Debugging Facilities? A Controlled Experiment on Loops and Streams with Specific Debuggers. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 485–497. doi:10.1109/ICSE48619.2023.00058

[53] Guido Salvaneschi, Sven Amann, Sebastian Proksch, and Mira Mezini. 2014. An Empirical Study on Program Comprehension with Reactive Programming. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '14)*. ACM, 564–575. doi:10.1145/2635868.2635895

[54] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. 2017. On the Positive Effect of Reactive Programming on Software Comprehension: An Empirical Study. *IEEE Transactions on Software Engineering* 43, 12 (2017), 1125–1143. doi:10.1109/TSE.2017.2655524

[55] Marius Schenkluhn, Michael Knierim, Francisco Kiss, and Christof Weinhardt. 2024. Connecting Home: Human-Centric Setup Automation in the Augmented Smart Home. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. ACM, 1–17. doi:10.1145/3613904.3642862

[56] Maximilian Schiedermeier, Jörg Kienzle, and Bettina Kemme. 2024. Give Me Some REST: A Controlled Experiment to Study Effects and Perception of Model-Driven Engineering with a Domain-Specific Language. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS '24)*. ACM, 1–12. doi:10.1145/3640310.3674080

[57] Dieter Schmalstieg and Tobias Höllerer. 2016. *Augmented Reality: Principles and Practice*. Addison-Wesley.

[58] Constantin Scholz, Hoang-Long Cao, Christian Skupin, Achim Ebert, and Martin Ruskowski. 2025. Sensor-Enabled Safety Systems for Human–Robot Collaboration: A Review. *IEEE Sensors Journal* 25, 2 (2025), 2488–2508. doi:10.1109/jsen.2024.3496905

[59] Mickael Sereno, Xiyao Wang, Lonni Besancon, Michael J. McGuffin, and Tobias Isenberg. 2022. Collaborative Work in Augmented Reality: A Survey. *IEEE Transactions on Visualization and Computer Graphics* 28, 6 (2022), 2530–2549. doi:10.1109/TVCG.2020.3032761

[60] Zohreh Sharafi, Bonita Sharif, Yann-Gaël Guéhéneuc, Andrew Begel, Roman Bednarik, and Martha Crosby. 2020. A Practical Guide on Conducting Eye Tracking Studies in Software Engineering. *Empirical Software Engineering* 25, 5 (2020), 3128–3174. doi:10.1007/s10664-020-09829-4

[61] Maulshree Singh, Jayasekara Kapukotuwa, Evert Lopes Gouveia, Eoin Fuenmayor, Yansong Qiao, Niall Murray, and Declan Devine. 2024. Unity and ROS as a Digital and Communication Layer for Digital Twin Application: Case Study of Robotic Arm in a Smart Manufacturing Cell. *Sensors* 24, 17 (2024), 5680. doi:10.3390/s24175680

[62] Jacques Smit and Herman Engelbrecht. 2024. Spatial Publish/Subscribe: Decoupling Game State Dissemination from State Computation for Massive Multiplayer Online Games. In *Proceedings of the 16th International Workshop on Massively Multiuser Virtual Environments (MMVE '24)*. ACM. doi:10.1145/3652212.3652216

[63] Kathleen Strodick and Tim Schattkowsky. 2024. Visual Scripting in Unity: A Comparative Analysis of Existing Frameworks. In *Proceedings of the IEEE/ACM 8th International Workshop on Games and Software Engineering (GAS '24)*. ACM. doi:10.1145/3643658.3643921

[64] Jeffrey Stylos, Benjamin Graf, Daniel K. Busse, Carsten Ziegler, Ralf Ehret, and Jan Karstens. 2008. A Case Study of API Redesign for Improved Usability. In *2008 IEEE Symposium on Visual Languages and Human-Centric Computing*. IEEE, 189–192. doi:10.1109/VLHCC.2008.4639083

[65] Jeffrey Stylos and Brad A. Myers. 2008. The Implications of Method Placement on API Learnability. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '08)*. ACM, 105–112. doi:10.1145/1453101.1453117

[66] Jose Ignacio Trasobares, Álvaro Domingo, Jorge Echeverría, Lorena Arcega, and Carlos Cetina. 2025. Deepening Our Understanding on the Use of Models and Code in Game Software Engineering: A Controlled Experiment in Unreal Engine. In *Proceedings of the ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS '25)*. IEEE. doi:10.1109/MODELS67397.2025.00016

[67] Unity Technologies. 2024. Unity Events Manual. https://docs.unity3d.com/Manual/unity-events.html Accessed: 2026-01-31.

[68] Unity Technologies. 2025. Unity Real-Time Development Platform. https://unity.com Version 6.3 LTS. Accessed: 2026-03-31.

[69] Tianyi Wang, Xun Qian, Fengming He, Xiyun Hu, Yuanzhi Cao, and Karthik Ramani. 2021. GesturAR: An Authoring System for Creating Freehand Interactive Augmented Reality Applications. In *Proceedings of the 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21)*. ACM, 552–567. doi:10.1145/3472749.3474769

[70] Haijun Xia, Sebastian Herscher, Ken Perlin, and Daniel Wigdor. 2018. Spacetime: Enabling Fluid Individual and Collaborative Editing in Virtual Reality. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST '18)*. ACM, 853–866. doi:10.1145/3242587.3242597

[71] Lei Zhang and Steve Oney. 2020. FlowMatic: An Immersive Authoring Tool for Creating Interactive Scenes in Virtual Reality. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. ACM, 481–493. doi:10.1145/3379337.3415824

[72] Celso Zimmerle and Kiev Gama. 2025. On the Usability of Reactive Programming APIs: A Mixed Evaluation. *Software: Practice and Experience* (2025). doi:10.1002/spe.3435

A Task Solution Code

This appendix presents the core messaging logic from each task's solution under both conditions. Starter scripts provided goal descriptions and infrastructure but no messaging logic; participants implemented the communication patterns shown below. Full solution files are available in the supplementary materials.

A.1 T1: Safety Zone Alerts

Key differences. Cascade uses three `Within()` calls with increasing radii; the responder's priority logic retains only the highest-severity alert. Events requires explicit iteration with per-indicator distance calculation and if/else-if branching. Both send zone changes upward (Cascade via `NotifyValue`; Events via direct method call).

A.2 T2: Alert Aggregation

Key differences. Cascade's auto-cascade propagates diagnostics downward without explicit forwarding; the participant only writes the upward handler. Events requires five distinct mechanisms: `GetComponentInParent`, `GetComponentsInChildren`, C# event subscription (`+=`), UnityEvent subscription (`.AddListener`), and explicit foreach forwarding.

A.3 T3: Drone Swarm Command

Key differences. Cascade dispatches to all drones with a single `BroadcastValue` call; ACKs propagate upward automatically. Events requires per-drone iteration, explicit unsubscribe/resubscribe to prevent duplicates, and separate method calls for dispatch and ACK handling. The routing graph diverges from the Transform hierarchy (Section 5.5).

B Correctness Rubric

An automated `CorrectnessChecker` script validated each submission against a deterministic test sequence. Binary pass/fail was supplemented by a partial-credit score (fraction of checks passed). The checker called `RefreshResponders()` on all relay nodes before each validation to ensure routing tables were initialized regardless of wiring method.

B.1 T1: Safety Zone Alerts

The checker moves the worker to three test positions and waits 1.5 s at each for state to settle:

(1) **Far** (>3.5 m from all indicators): all 8 indicators must show CLEAR.
(2) **Mid** (~1.9 m from nearest indicator): indicators within 2 m show WARNING, within 3.5 m show CAUTION, beyond show CLEAR.
(3) **Close** (~0.84 m from nearest indicator): indicators within 1 m show DANGER, within 2 m WARNING, within 3.5 m CAUTION, beyond CLEAR.

```
// Spatial alerts: three distance thresholds
relay.Send("danger").ToDescendants().Within(1f).Execute();
relay.Send("warning").ToDescendants().Within(2f).Execute();
relay.Send("caution").ToDescendants().Within(3.5f).Execute();
// Zone status to dashboard (upward)
float dist = Vector3.Distance(workerTransform.position, zoneCenter.position);
string zone = dist<=1f ? "DANGER" : dist<=2f ? "WARNING" : dist<=3.5f ? "
      CAUTION" : "SAFE";
if (zone != lastZone) { relay.NotifyValue($"Zone: {zone}"); lastZone = zone; }
```

Cascade: ZoneAlertManager (Update loop)

```
foreach (var ind in indicators) {
  if (ind == null) continue;
  float dist = Vector3.Distance(workerTransform.position, ind.transform.
      position);
  if (dist <= 1f) ind.HandleAlert("danger");
  else if (dist <= 2f) ind.HandleAlert("warning");
  else if (dist <= 3.5f) ind.HandleAlert("caution");
  else ind.HandleAlert("clear");
}
float zoneDist = Vector3.Distance(workerTransform.position, zoneCenter.
      position);
string zone = zoneDist<=1f ? "DANGER" : zoneDist<=2f ? "WARNING" : zoneDist
      <=3.5f ? "CAUTION" : "SAFE";
if (zone != lastZone && dashboard != null) { dashboard.AddAlert($"Zone: {zone}
      "); lastZone = zone; }
```

Unity Events: ZoneAlertManager (Update loop)

```
protected override void ReceivedMessage(MessageString msg) {
  if (relay == null) return;
  if (msg.value == "DIAGNOSTIC") return; // don't echo back up
  relay.NotifyValue($"[{subsystemName}] {msg.value}");
}
```

Cascade: SubsystemAlerter (message handler)

```
void Start() {
  dashboard = GetComponentInParent<CentralDashboard_Events>();
  sensors = GetComponentsInChildren<SensorNode_Events>();
  foreach (var s in sensors) s.OnSensorMessage += HandleSensorMessage;
  var b = GetComponentInParent<DiagnosticBroadcaster>();
  if (b != null) b.OnDiagnosticBroadcast.AddListener(OnDiagnosticCheck);
}
void HandleSensorMessage(string msg) {
  if (dashboard != null) dashboard.AddAlert($"[{subsystemName}] {msg}");
}
void OnDiagnosticCheck(string cmd) {
  foreach (var s in sensors) if (s != null) s.HandleDiagnostic(cmd);
}
```

Unity Events: SubsystemAlerter (Start + handlers)

```
public void SendMissionCommand() {
  if (relay == null) return;
  relay.BroadcastValue("EXECUTE");       // down to all drones
  relay.NotifyValue("Mission dispatched"); // up to display
}
protected override void ReceivedMessage(MessageString msg) {
  if (msg.value.StartsWith("ACK:") && relay != null)
    relay.NotifyValue(msg.value); // forward ACK upward
}
```

Cascade: CommandStation

```
void Start() {
  drones = transform.parent.GetComponentsInChildren<DroneUnit_Events>();
  display = GetComponentInParent<CentralDashboard_Events>();
}
public void SendMissionCommand() {
  foreach (var d in drones) {
    if (d == null) continue;
    d.OnAcknowledge -= HandleAck;
    d.OnAcknowledge += HandleAck;
    d.ReceiveCommand("EXECUTE");
  }
  if (display != null) display.AddAlert("Mission dispatched");
}
void HandleAck(string ack) { if (display != null) display.AddAlert(ack); }
```

Unity Events: CommandStation

Each indicator at each position is one check. The checker reads the indicator's current state (the highest-priority alert received, resolving any overlapping `Within()` calls in the Cascade condition). The dashboard must contain both a zone keyword (DANGER, WARNING, CAUTION, or SAFE) and the word "ZONE." Score = correct checks / total checks.

B.2 T2: Alert Aggregation

(1) **Upward aggregation** (15 s observation): the dashboard must contain alert entries from all four subsystems ([HVAC], [Electrical], [Hydraulics], [Cooling]).
(2) **Downward diagnostic**: the checker triggers a diagnostic broadcast, waits 3 s, then verifies each subsystem's "OK" response appears on the dashboard with the correct subsystem prefix.

Score = (subsystems with alerts + subsystems with OK responses) / 8.

B.3 T3: Drone Swarm Command

(1) **Command dispatch**: after calling `SendMissionCommand()`, all 6 drones must transition to ACTIVE state within 1 s.
(2) **Mission report**: the display log must contain "MISSION DISPATCHED."
(3) **ACK forwarding**: within 0.5 s, the display log must contain "ACK" entries for all 6 named drones (Alpha through Zeta).

Score = (active drones + mission log + drone ACKs) / 13.

C Training Reference Cards

Participants received an online reference card as a pinned browser tab. Full training tutorials are in the supplementary materials. **Pitfall warnings:** (1) Types are concrete, not generic. (2) `.value` is lowercase. (3) `Within()` measures from relay node position. (4) Routing tables must be wired for messages to propagate.

Table 2: Cascade reference card summary.

Category	Content
Setup	Extend `BaseResponder`; attach `RelayNode`; wire routing tables
Tier 1	`relay.BroadcastValue()` (down), `relay.NotifyValue()` (up)
Tier 2	`relay.Send(val).ToDescendants()` `.Within(r).Execute()`
Directions	`.ToChildren()`, `.ToDescendants()`, `.ToParents()`, `.ToAncestors()`, `.ToAll()`
Spatial	`.Within(r)`, `.InCone(...)`, `.InBounds(b)`
Receiver	Override `ReceivedMessage(MessageString msg)`; access `msg.value`
Types	`MessageBool`, `MessageInt`, `MessageFloat`, `MessageString`, etc. (concrete, not generic)

Table 3: Unity Events reference card summary.

Category	Content
Setup	Extend `MonoBehaviour`; find and cache components in `Start()`
Discovery	`GetComponentsInChildren<T>()`, `GetComponentInParent<T>()`, `[SerializeField]`
C# events	`source.OnEvent += Handler`
UnityEvents	`source.OnBroadcast.AddListener(Handler)`

Pitfall warnings: (1) If targets are siblings, search from `transform.parent`. (2) Finding ≠ subscribing. (3) Cache in `Start()`, not `Update()`. (4) Always null-check.

D Task Scene Hierarchies and Routing Graphs

Each task scene contains a Unity Transform hierarchy and, for Cascade, an explicit routing graph that determines message propagation. The routing graph may differ from the Transform hierarchy (as in T3). Figures below show both hierarchies; solid arrows indicate routing edges, and dashed boxes mark nodes with no relay (grouping only). Spatial thresholds are shown as concentric regions where applicable.

D.1 T1: Safety Zone Alerts

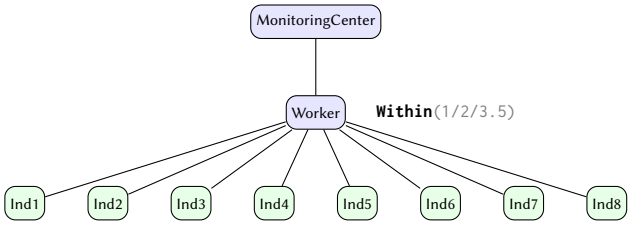


Figure 12: T1 routing graph. Worker dispatches spatial alerts to 8 indicators via `Within()` at three thresholds (1 m danger, 2 m warning, 3.5 m caution) and notifies zone status upward.

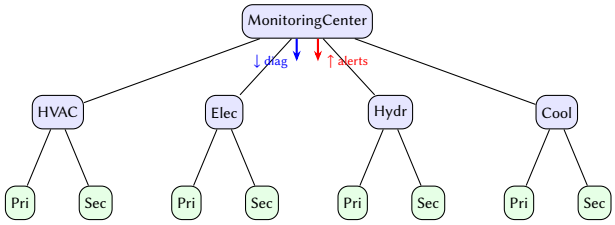


Figure 13: T2 routing graph (3 levels). Sensor alerts propagate upward (red); diagnostic commands cascade downward (blue). Each subsystem station relays in both directions.

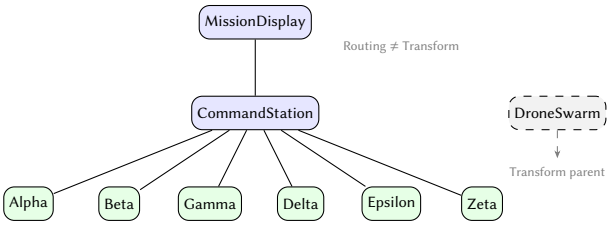


Figure 14: T3 routing graph (diverges from Transform hierarchy). Drones are Transform children of DroneSwarm (dashed) but routing children of CommandStation. Commands broadcast down; ACKs notify up.

D.2 T2: Alert Aggregation

D.3 T3: Drone Swarm Command

E Counterbalancing Design

Condition order (AB vs. BA) was counterbalanced across participants. Task order within each condition used a balanced Latin square with 3 orderings, crossed with condition order, yielding 6 counterbalancing groups.

Table 4: Counterbalancing assignment. AB = Cascade first; BA = Events first. Task orderings follow a balanced 3×3 Latin square.

Group	Cond. Order	Task Order	n
1	AB	T1, T2, T3	3
2	AB	T2, T3, T1	2
3	AB	T3, T1, T2	2
4	BA	T1, T2, T3	3
5	BA	T2, T3, T1	2
6	BA	T3, T1, T2	2

The same task order was used in both conditions for a given participant, as described in Section 5.7.

F Questionnaire Instruments

F.1 NASA-TLX

We administered an adapted NASA-TLX [28] (without pairwise weighting) after each condition's three tasks. Participants rated six subscales on 7-point Likert scales (1–7):

(1) **Mental Demand**: How mentally demanding were the tasks?
(2) **Physical Demand**: How physically demanding were the tasks?
(3) **Temporal Demand**: How hurried or rushed was the pace?

(4) **Performance**: How successful were you in accomplishing what you were asked to do? (1 = Perfect, 7 = Failure; reverse-scored)
(5) **Effort**: How hard did you have to work to accomplish your level of performance?
(6) **Frustration**: How insecure, discouraged, irritated, stressed, and annoyed were you?

The prompt specified "the three tasks you just completed using [Cascade / Unity Events]" to anchor responses to the specific condition.

F.2 System Usability Scale (SUS)

We administered SUS [5] per condition (after each condition's tasks and NASA-TLX). The standard 10 items were adapted to reference the specific messaging approach:

(1) I think that I would like to use [Cascade / Unity Events] frequently.
(2) I found [Cascade / Unity Events] unnecessarily complex.
(3) I thought [Cascade / Unity Events] was easy to use.
(4) I think that I would need the support of a technical person to be able to use [Cascade / Unity Events].
(5) I found the various functions in [Cascade / Unity Events] were well integrated.
(6) I thought there was too much inconsistency in [Cascade / Unity Events].
(7) I would imagine that most people would learn to use [Cascade / Unity Events] very quickly.
(8) I found [Cascade / Unity Events] very cumbersome to use.
(9) I felt very confident using [Cascade / Unity Events].
(10) I needed to learn a lot of things before I could get going with [Cascade / Unity Events].

Responses on a 5-point Likert scale (Strongly Disagree to Strongly Agree). Scoring follows the standard SUS formula [5].

G Roslyn Analyzer Catalog

Table 5 lists all 15 Roslyn analyzers included with Cascade. Two analyzers (CA005, CA010) include code fix providers that offer one-click resolution in the IDE.

H Full Benchmark Data

Table 6 reports per-invocation overhead (mean of runs 1–2; run 0 warmup excluded) for all benchmarked configurations. IL2CPP standalone build on Intel Core i9-12900K, 32 GB RAM, RTX 4090, Unity 6.3 LTS. Fisher-Yates shuffled method order; 2-second measurement windows; run-to-run variance 2–8%.

I Power Analysis Details

We conducted an a priori power analysis to determine the minimum sample size for our within-subjects design. Our primary analyses use linear mixed-effects models (LMMs) with condition, task, and their interaction as fixed effects and participant as a random intercept. Because closed-form power calculations for LMMs depend on assumptions about the variance structure that are unavailable before data collection, we first report power calculations for the simpler paired-samples t-test as a conservative lower bound, then present simulation-based LMM estimates.

I.1 Paired-Samples t-Test (Conservative Estimate)

For a two-tailed paired-samples t-test at $\alpha = 0.05$, we computed required sample sizes using G*Power 3 [16]. Table 8 shows required N and achieved power at $n = 14$ for various effect sizes.

Sensitivity analysis. At $n = 14$ with $\alpha = 0.05$ and power = 0.80, the minimum detectable effect size is $d = 0.80$ (two-tailed) or $d = 0.70$ (one-tailed). Effect sizes below $d = 0.80$ would require $n \geq 16$ for adequate power.

I.2 Effect Size Estimates from Prior Work

We base our power analysis on effect sizes from the most comparable prior study:

- **DSL effectiveness**: Johanson and Hasselbring [36] reported 31–56% time reductions ($n = 36$, within-subjects) comparing a DSL against C++ for ecological modeling tasks, with effect sizes of $d = 0.65$–0.81 for correctness and $d = 0.51$–0.64 for time across task types (computed from reported means and standard deviations).
- **API usability**: Stylos and Myers [65] reported 2–11× speedups ($n = 10$) for improved method placement but used nonparametric tests and did not report effect sizes.
- **Reactive paradigm comparison**: Salvaneschi et al. [53] found significant correctness differences ($p = 0.026$, Mann-Whitney U) between reactive and observer-pattern conditions ($n = 38$, between-subjects) but did not report effect sizes.

Based on Johanson and Hasselbring's reported $d = 0.86$–1.47 and the large speedup ratios observed in API usability studies, we conservatively assume $d = 0.80$ for our power calculations.

I.3 Multiplicity Correction

Our five confirmatory hypotheses group into two natural families based on the construct they measure. The *task performance* family (H1: correctness, H2: task completion) comprises two behavioral measures drawn from the same timed coding sessions; completion rate is effectively a binary threshold on the continuous correctness score, so these tests share a common underlying construct. The *subjective experience* family (H3: workload, H4: usability) comprises two self-report instruments administered after the same condition blocks, both capturing participants' perceived quality of the tool interaction. H5 (workflow focus) stands alone as a behavioral process metric (application-switching entropy) distinct from both task outcomes and subjective ratings.

We apply a Holm–Bonferroni procedure [32] within each family ($k = 2$), testing the smaller p at $\alpha/2 = 0.025$ and the larger at $\alpha = 0.05$. H5 is evaluated independently at $\alpha = 0.05$. As an additional sensitivity analysis, we report results under a global Holm–Bonferroni correction with $k = 5$; the most conservative step ($\alpha/5 = 0.01$) requires $d = 0.95$ at $n = 14$.

I.4 Simulation-Based LMM Power Analysis

We used the `simr` package for R [25] to estimate statistical power via Monte Carlo simulation of the LMM analysis. For each hypothesis, we constructed a model using `makeLmer` with the following assumed parameters:

Table 5: Complete catalog of Cascade Roslyn analyzers. Severity levels: Error, Warning, Info.

ID	Sev.	Title	Description	Fix
CA001	I	Consider using fluent DSL	Verbose legacy invocation with explicit metadata block could use DSL	
CA002	W	Self-only filter may be unintentional	Self-level filter used alone; child responders excluded	
CA003	W	Network message may not propagate	Invocation in network context but filter is Local	
CA004	I	Consider Broadcast convenience method	Pattern matches `BroadcastInitialize()`, `NotifyComplete()`, etc.	
CA005	W	Missing `.Execute()` call	Fluent chain built but never dispatched	✓
CA006	W	Missing `RefreshResponders()`	`AddComponent<Responder>()` without routing table refresh	
CA007	W	Potential infinite message loop	Handler sends same message type back to scope including self	
CA008	W	`SetParent` without routing update	Transform reparented but routing table not updated	
CA009	W	Override may need `base.Invoke()`	Invocation overridden without calling base	
CA010	E	`[GenerateDispatch]` requires partial	Source generator attribute on non-partial class	✓
CA011	I	Consider `ExtendableResponder`	Invocation override with >3 custom method cases	
CA012	W	Tag without `TagCheckEnabled`	Tag assigned but tag filtering silently disabled	
CA013	I	Responder may need `RelayNode`	Responder sends messages but has no relay reference	
CA014	W	Possible misspelled handler	Method name within Levenshtein distance 3 of correct handler	
CA015	W	Incorrect filter comparison	Bitwise flag compared with == instead of &	

Table 6: Per-invocation overhead (µs) for spatial fan-out ($D = 1$) across receiver counts. Objects placed randomly in 40×40 m area, spatial radius 5 m.

Approach	$N=4$	$N=8$	$N=16$	$N=32$	$N=64$
Cascade spatial primitives					
Cascade **`Within()`**	1.08	1.19	1.40	2.46	3.23
Cascade **`InCone()`**	1.18	1.23	1.58	2.08	3.10
Cascade **`Within()+Active()`**	1.09	1.22	1.53	2.42	3.57
Cascade `Nearest(5)`	2.57	2.88	3.58	4.87	7.64
Unity baselines					
Uncached GCIC + distance	1.84	2.06	2.58	4.12	5.99
Physics.OverlapSphereNonAlloc	1.19	1.23	1.18	2.58	2.75
Cached manual + distance	0.43	0.56	0.76	1.35	2.14
Cached manual + active + dist.	0.58	0.70	1.00	1.73	2.90
Cached manual cone	0.40	0.47	0.66	1.12	1.91
Manual sort nearest-K	0.71	0.92	1.08	1.58	2.42
C# delegate + distance	0.39	0.51	0.72	1.34	2.19
UnityEvent + distance	0.52	0.63	0.83	1.37	2.48

Table 7: Depth sweep: per-invocation overhead (µs) for spatial fan-out with branching factor 4 ($N = \mathbf{bf}^D$).

Approach	$D=1$ ($N=4$)	$D=2$ ($N=16$)	$D=3$ ($N=64$)
Cascade **`Within()`**	1.08	1.54	4.96
Cascade **`InCone()`**	1.18	1.75	4.89
Cascade **`Within()+Active()`**	1.09	1.66	5.04
Uncached GCIC + distance	1.84	2.43	5.62
Physics.OverlapSphereNonAlloc	1.19	1.27	2.79
Cached manual + active + dist.	0.58	0.89	2.58
C# delegate + distance	0.39	0.67	1.82
UnityEvent + distance	0.52	0.79	2.12

- **Design**: 2 conditions × 3 tasks × N participants (within-subjects), yielding $6N$ observations per model.
- **Fixed effect**: Condition coefficient set to the expected Cohen's d from prior work.
- **Random intercept**: Participant variance = 0.4286 (ICC = 0.30), reflecting typical between-participant variability in within-subjects HCI studies.
- **Residual SD**: 1.0 (standardized units).
- **Simulations**: 1,000 Monte Carlo replications per cell.

The simulation confirms that $n = 14$ provides adequate power ($\geq 89\%$) for all five confirmatory hypotheses at $d = 0.80$. H1 (correctness) benefits from the LMM's repeated-measures structure (3 tasks × 2 conditions = 6 observations per participant), achieving $\geq 97\%$ power. H3–H5 are per-condition summary measures (one score per

Table 8: Required n for paired-samples t-test ($\alpha = 0.05$, two-tailed, 80% power) and achieved power at n=14.

Effect Size	d	Required n	Power at n=14
Small	0.20	196	0.09
Small-Medium	0.35	64	0.20
Medium	0.50	32	0.46
Medium-Large	0.65	19	0.72
Large	0.80	13	0.89
Very Large	1.00	8	0.97

Table 9: Simulation-based power (uncorrected $\alpha = 0.05$, 1,000 simulations) for confirmatory hypotheses. H1 uses LMM; H2 uses GLMM (logit); H3–H5 use paired t-test/Wilcoxon (one observation per participant per condition).

Hyp.	Measure	d	N=12	N=14	n=20
H1	Correctness	0.80	95.2	97.5	99.8
H2	Task completion	0.80	95.2	97.5	99.8
H3	NASA-TLX	0.80	86.0	89.0	95.0
H4	SUS	0.80	86.0	89.0	95.0
H5	Switching entropy	0.80	86.0	89.0	95.0

Table 10: Sensitivity analysis: power with joint Holm–Bonferroni correction ($k = 5$, 1,000 simulations).

Hyp.	Measure	d	n=12	N=14	N=20
H1	Correctness	0.80	90.5	94.0	99.0
H2	Task completion	0.80	90.5	94.0	99.0
H3	NASA-TLX	0.80	78.0	82.0	91.0
H4	SUS	0.80	78.0	82.0	91.0
H5	Switching entropy	0.80	78.0	82.0	91.0
Minimum power			78.0	82.0	91.0

Table 11: Power comparison at n=14 (uncorrected $\alpha = 0.05$). H1 benefits from LMM's repeated-measures structure; H3–H5 use paired tests (one score per condition per participant).

Hyp.	Measure	d	Paired t/Wilcoxon	LMM
H1	Correctness	0.80	89.0%	97.5%
H2	Task completion	0.80	89.0%	97.5%
H3	NASA-TLX	0.80	89.0%	—
H4	SUS	0.80	89.0%	—
H5	Switching entropy	0.80	89.0%	—

participant per condition) analyzed with paired tests, achieving 89% power. H2 (task completion rate) is analyzed with a GLMM (logit link); power for binary outcomes depends on observed completion rates and may differ from the LMM estimates above. As a sensitivity analysis, under Holm-Bonferroni correction ($k = 5$), the minimum power at $n = 14$ is 82% (H3–H5), still above the conventional 80% threshold.

I.5 Assumptions and Limitations

- **ICC assumption**: We assumed ICC = 0.30. If the true ICC is substantially higher, power decreases; a sensitivity analysis at ICC = 0.50 reduces power by approximately 5–10 percentage points.
- **Effect size transferability**: Expected effect sizes are drawn from prior DSL [36] and API usability [65] studies with different populations and tools; actual effects may differ.
- **Simplified simulation model**: The simulation used `y ~ condition + (1 | participant)`, omitting task as a fixed effect. Including task would partition additional variance, potentially *increasing* power.
- **Normal residuals**: The simulation assumes normally distributed residuals. For bounded outcomes (e.g., correctness scores in $[0, 1]$), we fit beta regression as a sensitivity analysis.

The final analyzed sample is $n = 14$.

Observed effects vs. a priori assumptions. The observed correctness effect ($d = 1.47$) and NASA-TLX effect ($d = 0.89$) both exceeded the assumed $d = 0.80$, confirming adequate power for H1 and H3. The SUS effect ($d = 0.58$) was below $d = 0.80$ but still reached significance ($p = .032$); at this effect size, our sample achieves approximately 46% power, so the SUS finding should be interpreted with caution and replicated in future work.

I.6 Eye Tracking Statistical Power

Eye tracking at 120 Hz yields approximately 57,600 pupil diameter samples per 8-minute task per participant. While individual samples are autocorrelated, the high temporal resolution enables per-fixation analysis (100–300 fixations per task, yielding 4,200–12,600 fixation events per condition across 14 participants and 3 tasks) and AOI dwell proportion analysis (210 observations per condition). We treat all eye tracking measures as exploratory and report uncorrected p-values with effect sizes and confidence intervals, following recommendations for physiological measures in software engineering research [60].